وزارة الاتصالات وتكنولوجيا المعلومات

رواد مصر الرقمية

Digital Egyptian Pioneers innitiative - DEPI

AI and Data Science - Machine learning track

BankWise

System Analysis & design

Eng: Abdelrahman

1. Problem Statement & Objectives

1.1 Problem Statement

Retail banking customers frequently need information about banking products, loan eligibility, interest rates, fees, required documents, and banking procedures. Obtaining this information through traditional channels, such as bank branches and customer service hotlines, can involve waiting times, limited availability, and repetitive interactions.

This creates additional workload for customer support teams, particularly when they repeatedly handle routine questions that could be answered using existing banking documentation.

Furthermore, customers may experience difficulty finding accurate and up-to-date information across different documents and sources. This challenge can be more significant for customers who prefer communicating in Arabic or using natural language rather than searching through formal banking documents.

For Banque Misr, these challenges create an opportunity to improve customer access to banking information through an intelligent, automated assistant.

BankWise AI addresses this problem by developing AI-powered virtual banking assistant that provides customers with accessible and reliable banking information. The system combines Retrieval-Augmented Generation (RAG) to retrieve information from verified banking documents with deterministic rule-based calculations for financial operations such as car-loan eligibility and installment estimation.

Unlike a general-purpose chatbot, BankWise AI is designed to ground its responses in approved banking information and isolate financial calculations from generative AI to reduce incorrect answers and numerical hallucinations.

1.2 Project Objectives

The main objectives of BankWise AI are:

- **Automate Customer Inquiries:** Provide an automated assistant capable of answering common Banque Misr banking inquiries without requiring customers to contact a branch or customer service representative for every routine question.
- **Implement Retrieval-Augmented Generation:** Develop a RAG pipeline that retrieves relevant information from verified Banque Misr documents and uses the retrieved content to generate contextually accurate responses.
- **Provide Reliable Loan Calculations:** Implement a deterministic loan calculation and eligibility engine that applies predefined financial rules and mathematical formulas rather than relying on the LLM to perform calculations.

- R**educe AI Hallucinations:** Restrict banking-related answers to relevant retrieved information and provide an appropriate fallback when sufficient supporting information is unavailable.
- **Maintain Data Accuracy and Freshness:** Organize banking information into structured and searchable knowledge sources, with a mechanism for updating stored information and displaying its last-updated date.
- **Improve Customer Experience:** Provide fast, understandable, and consistent answers through a conversational interface that supports follow-up questions.
- **Maintain Security and Responsible AI Practices:** Protect customer information, avoid exposing confidential data, and ensure that the assistant does not present estimates or general information as official loan approvals or financial decisions.

1.3 Project Scope

The initial implementation focuses on Banque Misr and the following capabilities:

- Frequently asked questions about banking services and procedures.
- Retrieval of information from banking policies and service documents.
- Car-loan program information and eligibility-related inquiries.
- Estimated monthly installment and total repayment calculations.

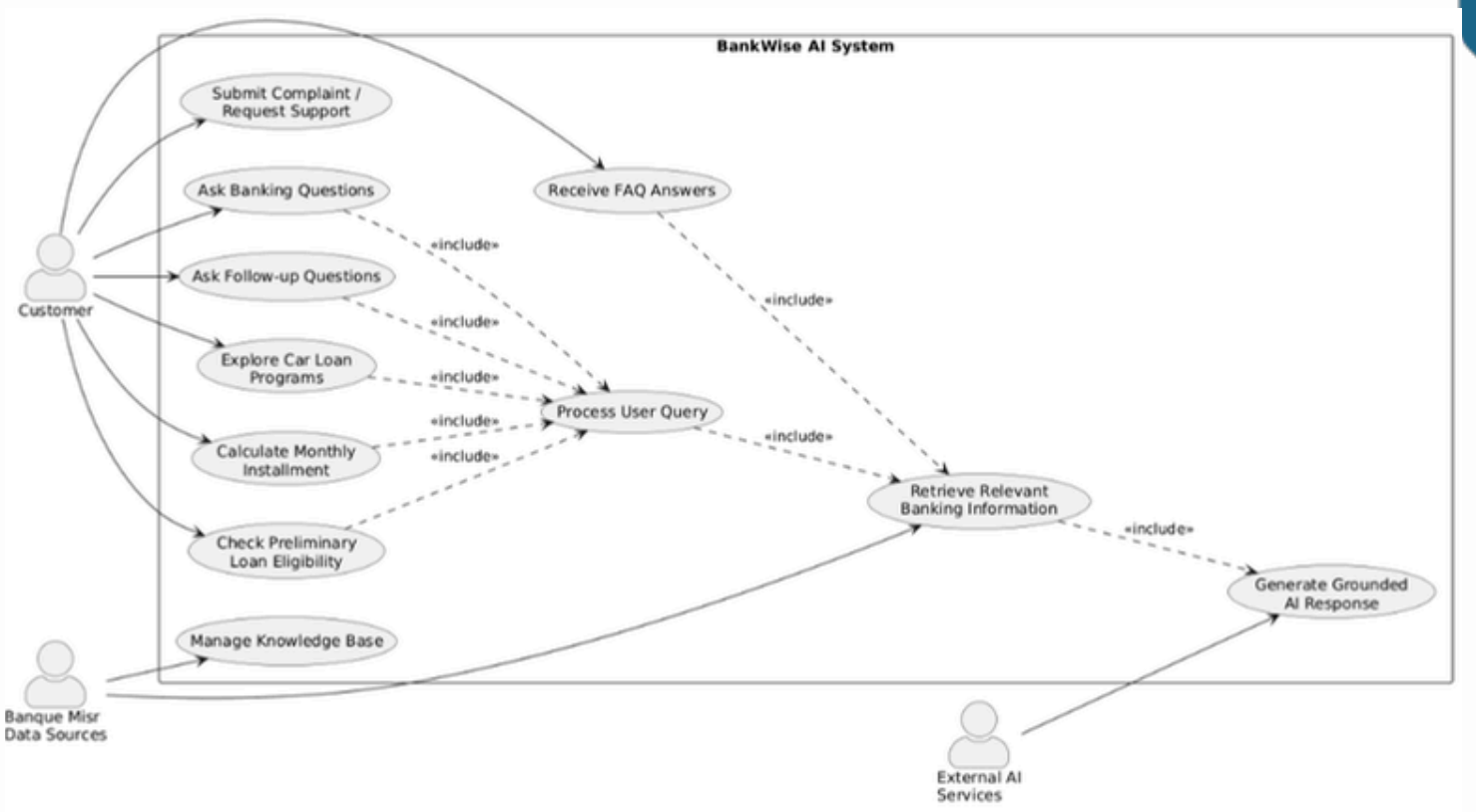
The system is intended to provide informational assistance and preliminary calculations. It does not replace official bank approval, customer verification, or direct banking transactions.

2. Use Case Diagram & Descriptions

2.1 System Actors

Actor	Description	Main Interactions
Customer	A Banque Misr customer or potential customer using the assistant.	Ask banking questions, request loan information, provide financial inputs, and receive answers.
Banque Misr Data Sources	Official banking documents and authorized API endpoints.	Provide banking policies, FAQ information, and loan-program parameters.
AI Services	External or locally hosted AI models and embedding services.	Generate embeddings, interpret queries, and generate grounded responses.

2.2 Use Case Diagram



Use Case	Description	Expected Result
UC-01: Ask Banking Questions	Customers ask questions in Arabic or English about Banque Misr services, fees, and procedures. The RAG system retrieves relevant information to generate a grounded answer.	Accurate answers based on available banking documents.
UC-02: Ask Follow-up Questions	Customers ask related questions while the system maintains the conversation context.	Relevant responses without repeating previous information.
UC-03: Request Car-Loan Information	Customers request details about available car-loan programs, including limits, durations, and interest rates.	Information about relevant loan programs.
UC-04: Calculate Loan Installment	Customers provide the loan amount and duration. The deterministic loan engine validates the inputs and calculates the estimated installment and total repayment.	An estimated repayment breakdown.
UC-05: Check Preliminary Loan Eligibility	Customers provide the required financial information, which is evaluated against predefined eligibility rules.	A preliminary eligibility result, not official loan approval.
UC-06: Manage Knowledge Base	Administrators update banking documents and structured data used by the RAG system.	An updated and searchable knowledge base.

Functional and non functional requirements

1- Chat : FR

Requirements
The system shall provide a chat interface where the customer types a question in Egyptian Arabic and receives a response.
The system shall always respond in Arabic. If the customer writes in another language, it shall politely state that the service is currently Arabic-only.
The system shall keep conversation context within a session, so follow-up questions can refer to earlier messages.
The system shall allow the customer to start a new conversation and clear the current session.

2- Loan Eligibility : FR

Requirements
The system shall extract the required inputs from the customer’s message: monthly income, existing monthly debts, loan amount, tenor, and age
If any input is missing or unclear, the system shall ask the customer for it before running the check.
The system shall validate inputs (numeric, positive, within sensible ranges) and reject invalid values with a clear message.
The system shall evaluate eligibility using a rule-based tool that applies rules loaded from a configuration file (debt burden ratio, minimum income, maximum tenor, age limits).
The system shall calculate the estimated monthly installment.
The system shall return the decision (eligible or not), the specific reasons, and the estimated installment.

3- FAQ : FR

Requirements
The system shall answer questions about loans, fees, and products using retrieved passages from the knowledge base of bank documents.
The system shall base answers only on retrieved content.
If no sufficiently relevant content is found, the system shall say it has no confirmed information and suggest contacting the bank.
The system shall allow the customer to start a new conversation and clear the current session.

Non Functional Req.

Category	Requirements
Accuracy	The system shall answer at least 8 of 10 representative test questions correctly.
Reliability	The system shall produce zero unhandled exceptions across the scripted test conversations, including empty, very long, or unusual input.
Error handling	When the LLM, a tool, or retrieval fails, the user shall receive a clear fallback message instead of a crash or technical error.
Privacy	The system shall not ask for or store sensitive personal data (national ID, card numbers), and session data shall not persist after the session ends.
Transparency	Each knowledge-base document shall have its source URL and retrieval date recorded.

3. AI System Architecture

3.1 Architecture Overview

BankWise AI follows a modular, service-oriented AI architecture. Its components are separated according to their responsibilities, allowing the conversational interface, language processing, information retrieval, and financial calculations to operate independently while communicating through defined interfaces.

The architecture combines three main approaches:

- **Retrieval-Augmented Generation (RAG):** Used to retrieve verified information from Banque Misr documents and generate grounded responses.
- **Agent-Based Query Processing:** Used to identify the customer's intent, determine which capabilities are required, and coordinate multi-step requests.
- **Deterministic Rule-Based Processing:** Used for financial calculations and eligibility assessments that require predictable mathematical and logical results.

The architecture is designed to prevent the LLM from independently inventing banking policies or performing critical financial calculations.

3.2 Main Architecture Components

A. User Interface Layer

Provides the interaction point between customers and the assistant.

Main responsibilities:

- Accept customer inquiries
- Display generated responses and calculation results.
- Support follow-up questions.
- Collect financial information required for loan calculations.
- Display relevant disclaimers and source information.
-

B. Backend and Request Management Layer

Acts as the communication layer between the user interface and AI components.

Main responsibilities:

- Receive and validate incoming requests.
- Manage conversation sessions.
- Route requests to the appropriate AI capability.
- Coordinate responses from the RAG engine and loan calculation engine.

C. AI Agent and Intent Classification Layer

Identifies the purpose of each customer inquiry and determines the appropriate processing path.

Possible intents include:

- General banking FAQ.
- Product and service information.
- Car-loan inquiry.
- Loan installment calculation.
- Preliminary loan eligibility.

For compound requests, this layer can divide the inquiry into smaller tasks and coordinate their execution.

D. RAG Knowledge Retrieval Layer

Provides access to verified banking information.

Its main processing stages are:

- **Document Ingestion:** Load banking documents and structured FAQ data.
- **Text Extraction:** Extract relevant content from PDF and other supported formats.
- **Preprocessing:** Normalize Arabic and English text and remove irrelevant content.
- **Chunking:** Divide documents into meaningful retrieval units while preserving context.
- **Embedding Generation:** Convert document chunks into numerical vector representations using an Arabic-capable multilingual embedding model.
- **Vector Storage:** Store embeddings, text, and associated metadata in a vector database.
- **Retrieval:** Search for relevant information based on the customer's query.
- **Grounded Generation:** Supply retrieved passages to the LLM to generate a supported response.

The system should retain document names, categories, language, and update information to support traceability.

E. Loan Calculation and Eligibility Engine

Handles financial operations independently of the LLM.

Main responsibilities:

- Retrieve the selected loan program's parameters.
- Validate customer-provided financial inputs.
- Apply configured eligibility rules.
- Calculate estimated monthly installments.
- Calculate total repayment and estimated interest.
- Return structured results with explanations.

The calculation engine uses deterministic Python logic rather than asking the LLM to perform financial mathematics.This is done using Banque Misr car-loan calculator API. It is a potential source of program parameters, including minimum and maximum loan amounts, duration limits, and annual interest rates. Its access permissions and data freshness must be verified before integration.

F. Large Language Model Layer

Responsible for understanding natural-language inquiries and generating conversational responses.

Main responsibilities:

- Interpret customer questions.
- Understand retrieved information and structured calculation results.
- Generate clear Arabic responses.
- Explain the reasons behind preliminary eligibility results.
- Follow system instructions and response restrictions.
- Avoid presenting unsupported information as confirmed banking facts.

The LLM does not independently determine official loan approval or replace the deterministic calculation engine.

G. Data Storage Layer

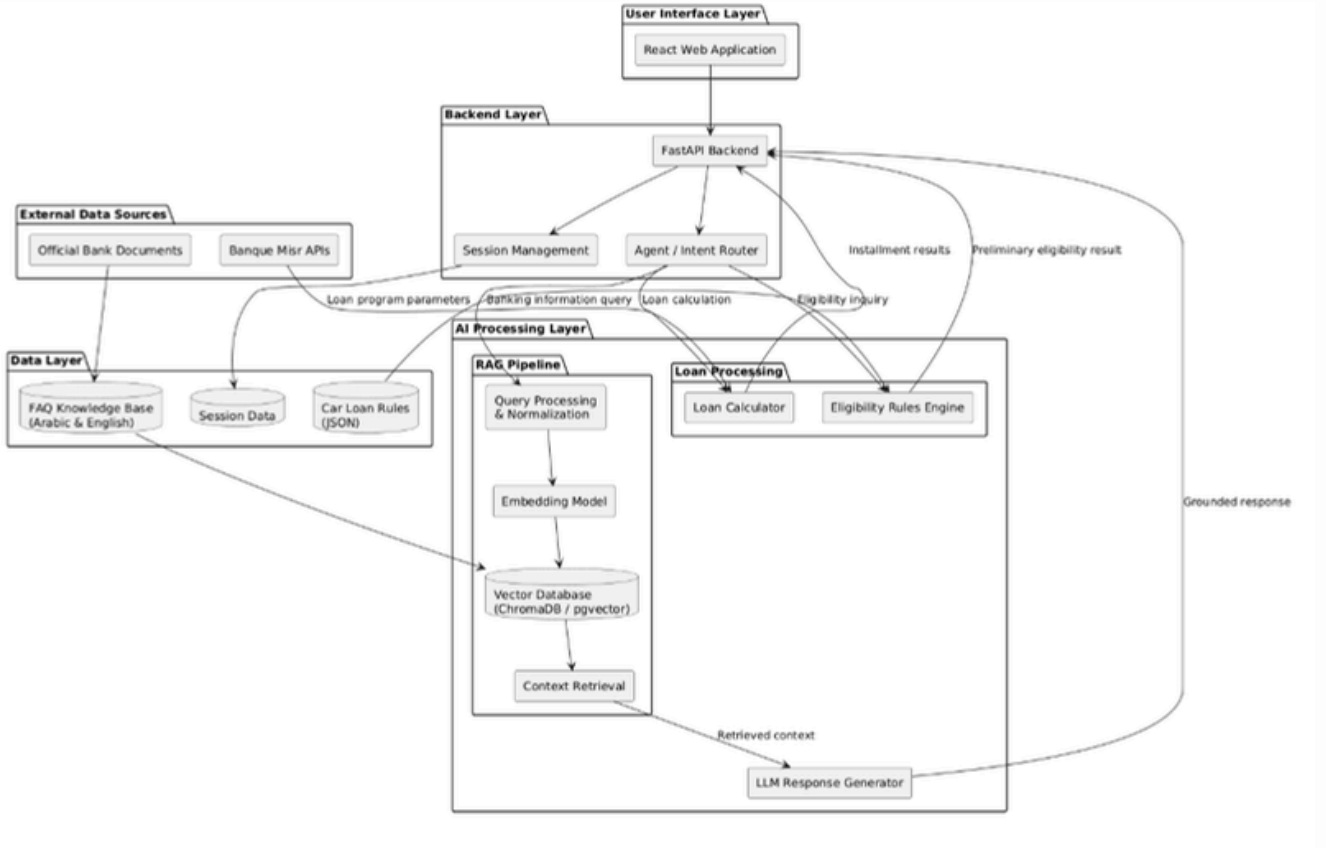
Maintains the information required by the system.

The proposed storage responsibilities include:

- **Vector Database:** Stores document embeddings, text chunks, and retrieval metadata.
- **Structured Data Storage:** Stores loan rules, program parameters, FAQs, and other structured information.
- **Session Storage:** Maintains temporary conversation context.
- **Monitoring Data:** Stores relevant operational metrics and usage information.

ChromaDB is a suitable vector database candidate for the prototype because of its integrated handling of vectors, documents, and metadata.

3.3 High-Level Architecture Diagram



3.4 Architecture Workflow

When a customer submits a question, the system follows this general process:

1. The frontend sends the customer's message to the backend.
2. The backend validates the request and retrieves the relevant session context.
3. The intent classification or agent layer determines the type of inquiry.
4. The request is routed to the appropriate processing component:
   - General banking questions are sent to the RAG pipeline.
   - Loan calculations are sent to the deterministic loan engine.
   - Complaint-related inquiries are sent to the complaint classification and routing component.
5. The selected component retrieves the necessary information or performs the required calculation.
6. The LLM generates a natural-language response using the retrieved information or structured results.
7. The backend returns the response to the frontend.
8. The system records relevant operational information for monitoring and future evaluation.

3.5 Proposed Technology Stack

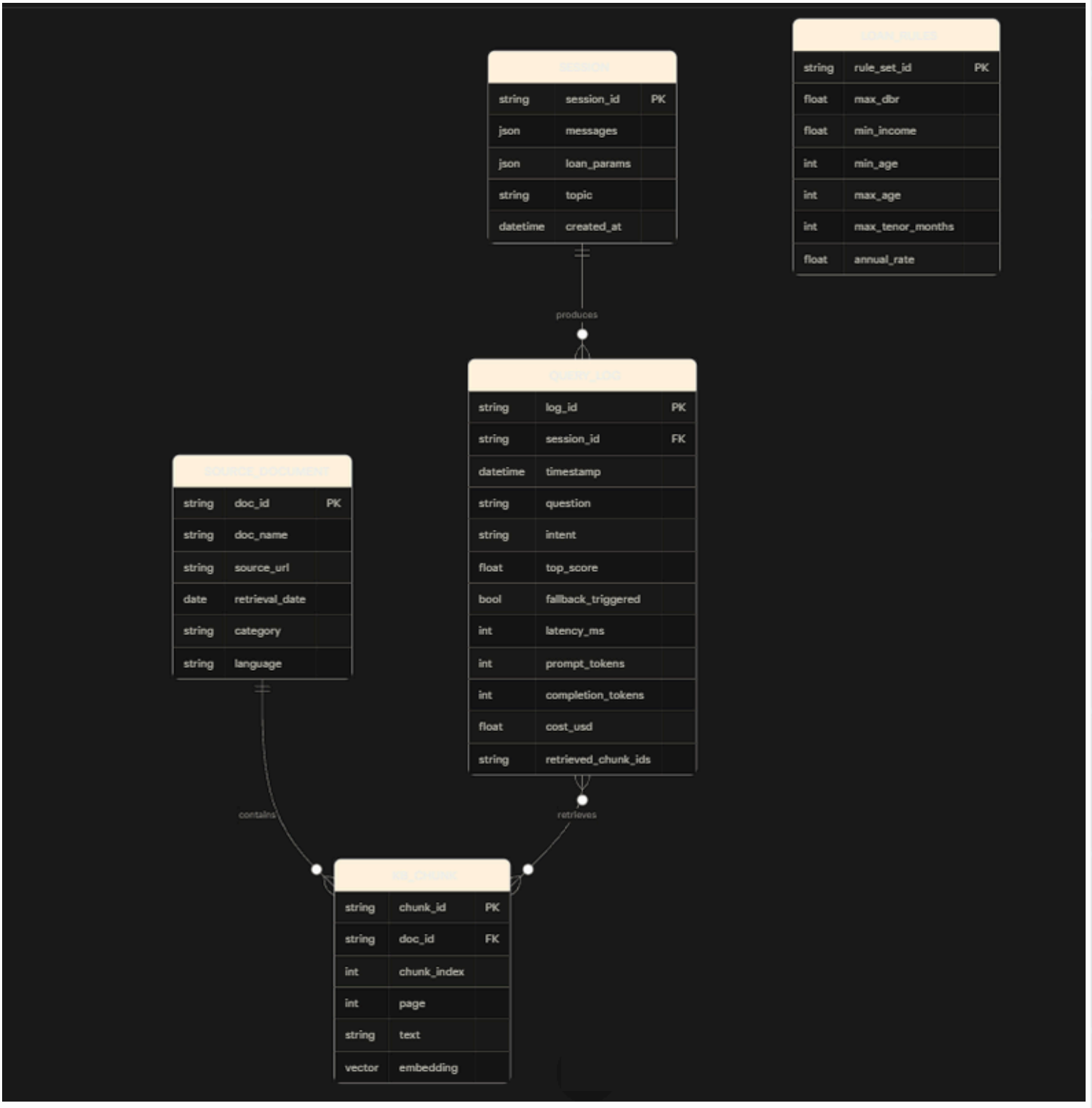
Component	Proposed Technology / Approach	Purpose
Frontend	React	Customer chat interface and administration dashboard
Backend	Python FastAPI	API development and request coordination
Agent Processing	Python-based agent and routing logic	Intent detection and task coordination
RAG Framework	Python RAG pipeline	Document ingestion, retrieval, and grounded generation
Embedding Model	Arabic-capable multilingual embedding model	Represent Arabic and English text as vectors
Vector Database	ChromaDB for the initial prototype	Store and retrieve document embeddings and metadata
Structured Data	JSON and relational storage where needed	Maintain FAQs, loan rules, and program information
LLM	An instruction-following language model through an API	Natural-language understanding and response generation
Financial Logic	Python deterministic calculation and rule engine (Banque Misr API)	Loan calculations and preliminary eligibility assessment
Deployment	Docker	Package and deploy application services

3.6 Design Principles

The architecture follows these principles:

1. **Modularity:** Each major component has a defined responsibility and can be developed or tested independently.
2. **Grounded Responses:** Banking information must be supported by retrieved knowledge sources.
3. **Deterministic Financial Processing:** Financial calculations and rule-based decisions must remain separate from generative AI.
4. **Data Traceability:** Responses should be traceable to their underlying documents or structured sources.
5. **Data Freshness:** Knowledge sources must have identifiable update dates and a controlled refresh process.
6. **Security and Privacy:** Customer information and system credentials must be protected.
7. **Scalability:** The modular design should support future expansion in users, banking products, and knowledge sources.

3.7 Data base design



Logical and physical schema

1. Storage mapping

Entity	Physical storage	Notes
SOURCE_DOCUMENT	`sources.json` (manifest)	Master record, one entry per source document
KB_CHUNK	ChromaDB collection `bankwise_kb`	Cosine distance, one embedding model per collection
SESSION	Redis key `session:{session_id}` with TTL	Ephemeral, deleted on reset or expiry
QUERY_LOG	`query_logs.jsonl`	Read by the admin dashboard
LOAN_RULES	`loan_rules.yaml`	Single config file reviewed by credit risk

2. SOURCE_DOCUMENT

Attribute	Type	Key	Description
doc_id	string	PK	Unique document id
doc_name	string		Document or page title
source_url	string		Origin URL (K-07)
retrieval_date	date		Date the content was collected (K-07)
category	string		loans / cards / accounts / fees
language	string		ar / en

3. KB_CHUNK (Chroma collection)

Chroma field	Attribute	Type	Key	Description
id	chunk_id	string	PK	`{doc_id}_{chunk_index}`
document	text	string		Normalized Arabic chunk text
embedding	embedding	float[]		BGE-M3 (1024 dims) or GATE-AraBERT (768 dims)
metadata	doc_id	string	FK	References SOURCE_DOCUMENT
metadata	chunk_index	int		Position inside the document
metadata	page	int		Page number if the source is a PDF
metadata	doc_name, source_url, retrieval_date, category	string		Copied from SOURCE_DOCUMENT (see normalization notes)

4. LOAN_RULES

Attribute	Type	Description
rule_set_id	string	Version label of the rule set
max_dbr	float	Maximum debt burden ratio
min_income	float	Minimum monthly income
min_age, max_age	int	Age limits
max_tenor_months	int	Maximum loan duration
annual_rate	float	Rate used in the installment formula

Data Flow & System Behavior

3.1 Data Flow Diagrams (DFD)

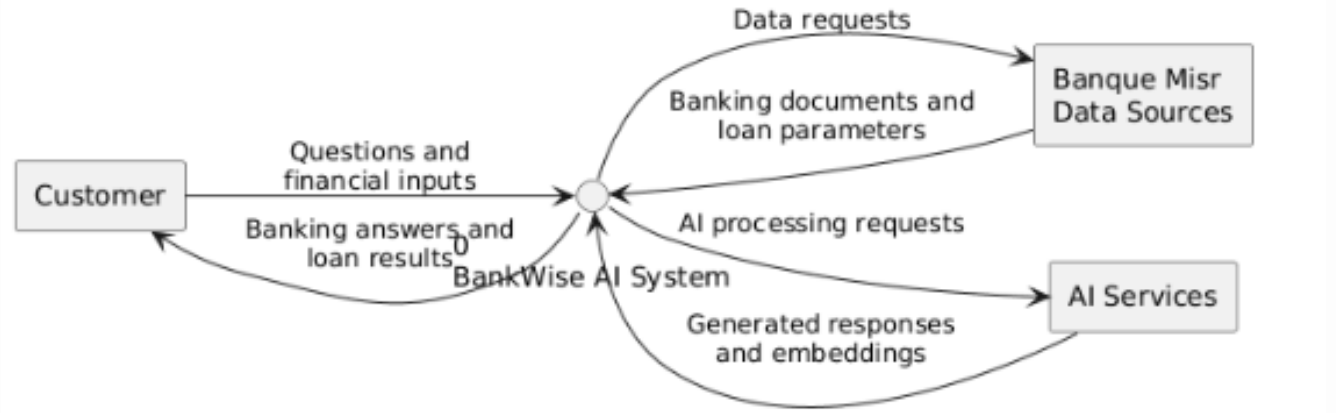
Data Flow Diagrams illustrate how information moves between the customer, BankWise AI components, and external banking data sources.

3.1.1 Context-Level DFD

The context-level DFD represents BankWise AI as one main process and shows its interaction with external entities.

External Entities:

- Customer: Submits banking inquiries and financial information and receives responses.
- Banque Misr Data Sources: Provide banking documents, FAQ information, and loan-program parameters.
- AI Services: Provide language-model and embedding capabilities.



- ### 3.1.2 Level 1 DFD

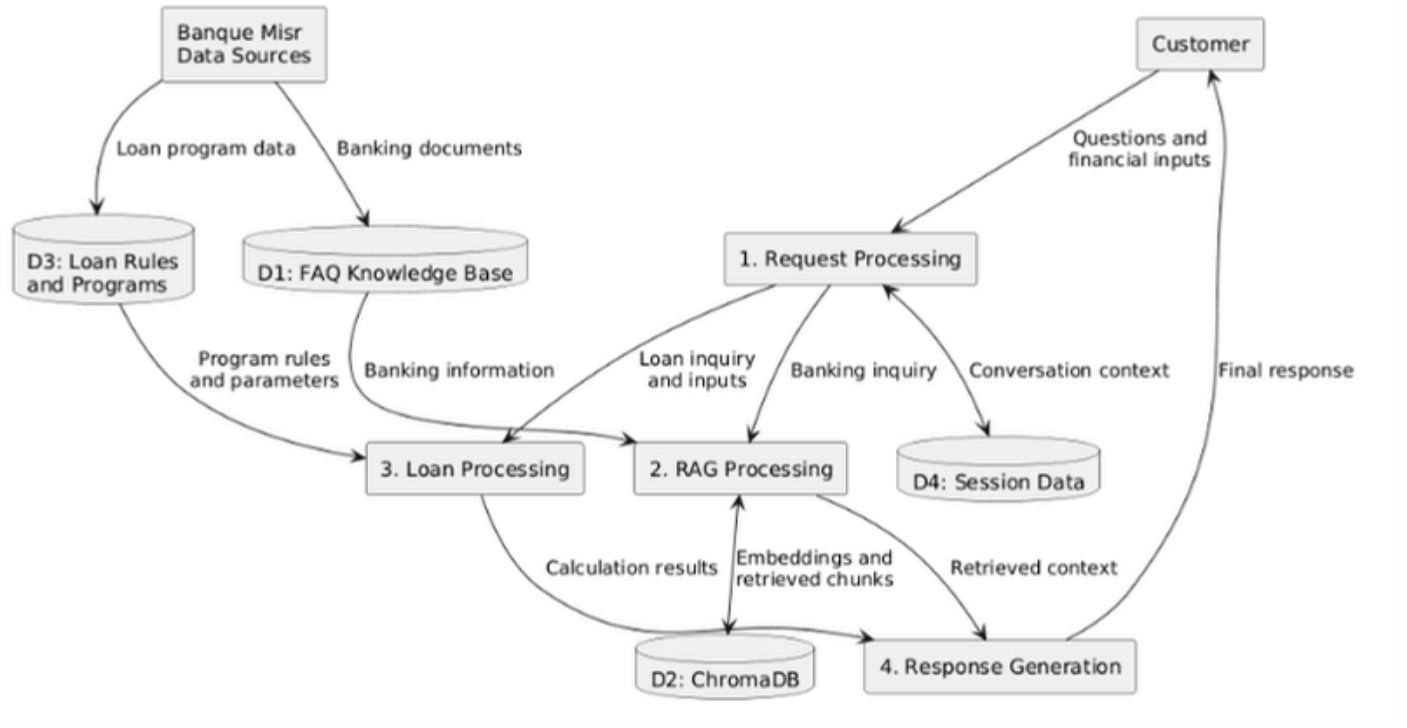
The Level 1 DFD decomposes BankWise AI into its main processing components and illustrates how data moves between them.

Main Processes:

- **Request Processing:** Receives customer messages and identifies the inquiry type.
- **RAG Processing:** Retrieves relevant information from banking knowledge base.
- **Loan Processing:** Validates financial inputs and performs loan calculations or preliminary eligibility assessments.
- **Response Generation:** Uses retrieved information or calculation results to generate the final response.

Data Stores:

- **D1 – FAQ Knowledge Base:** Contains structured banking FAQs and information.
- **D2 – ChromaDB:** Stores document embeddings, text chunks, and metadata.
- **D3 – Loan Rules and Programs:** Contains structured loan rules and program parameters.
- **D4 – Session Data:** Maintains conversation context.



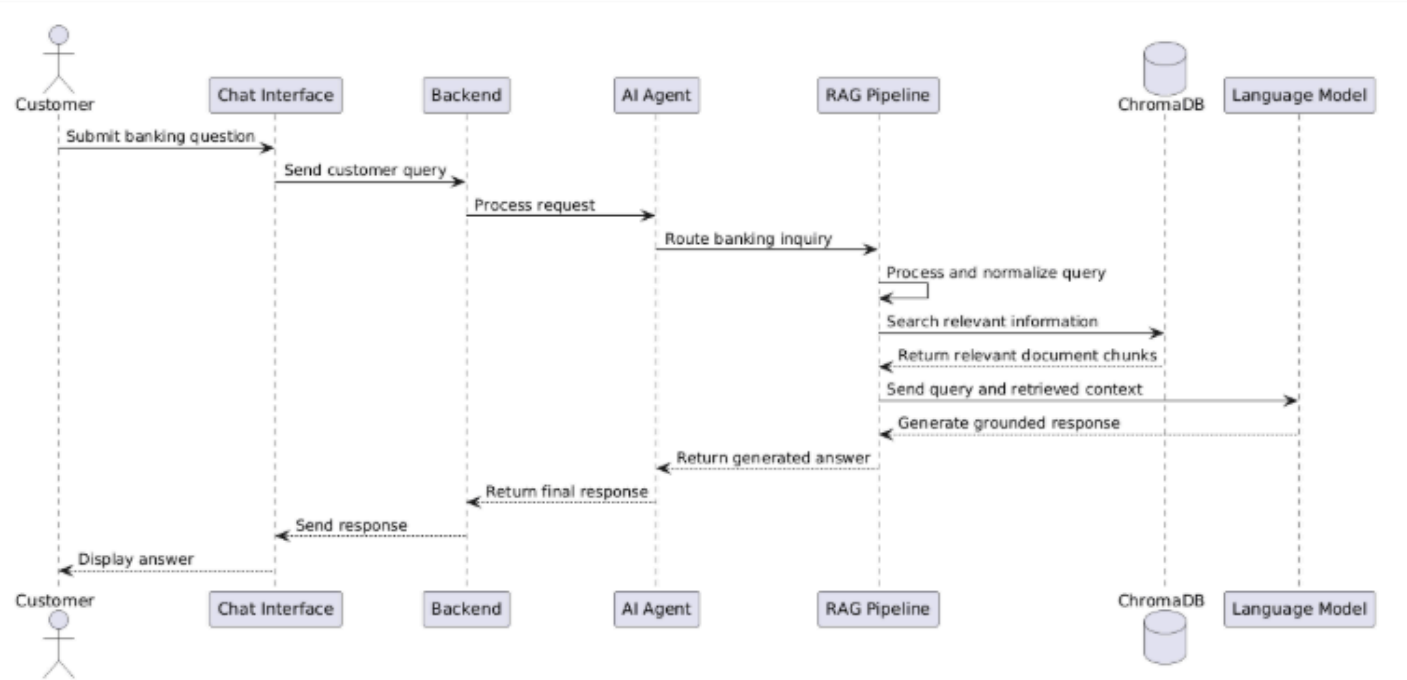
The RAG process is used for information-based inquiries, while the loan processing component handles financial calculations independently. The response generation component combines the appropriate results into a customer-facing answer

3.2 Sequence Diagrams

Sequence diagrams illustrate how the system components communicate during important customer interactions.

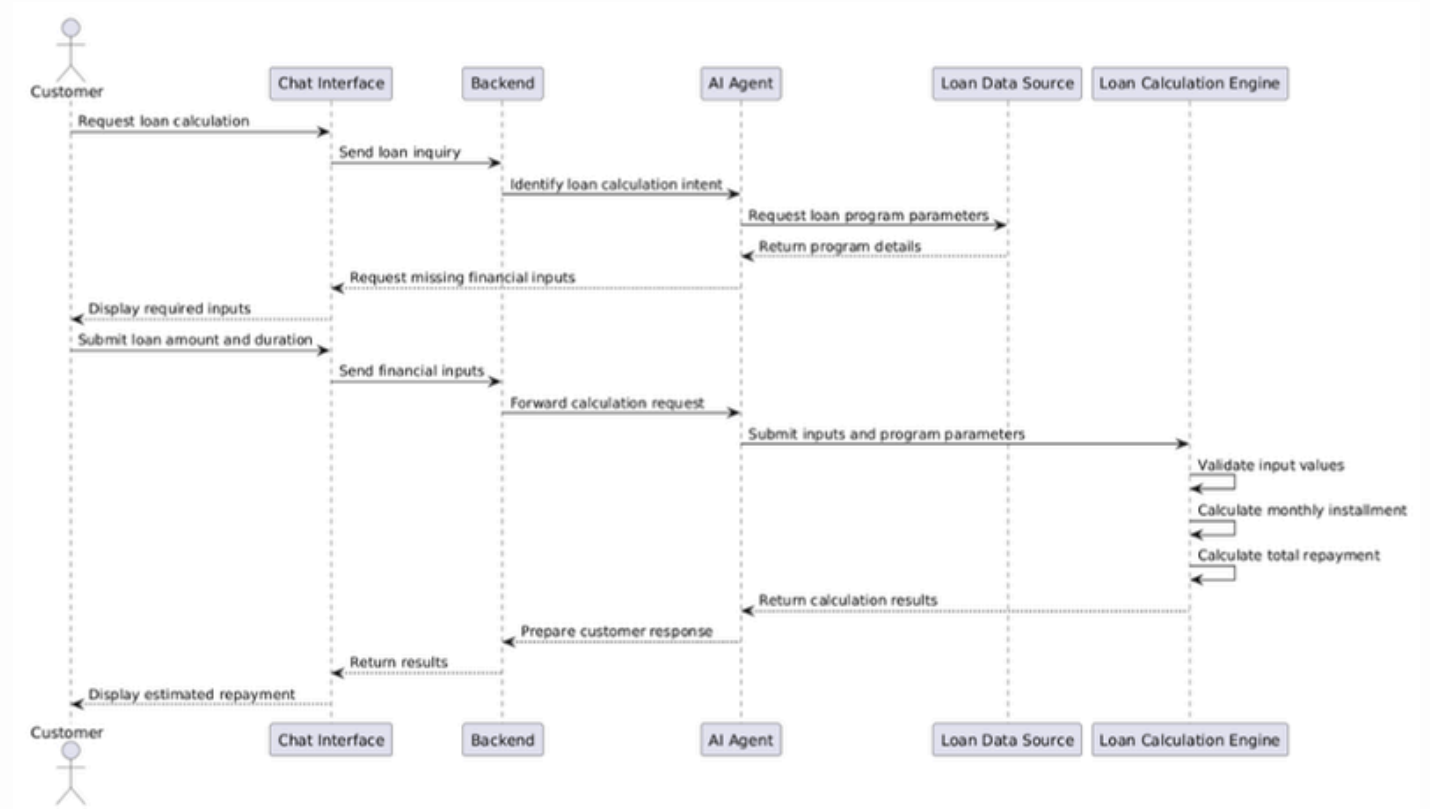
3.2.1 Banking Question Sequence

This sequence represents the process of answering a general banking inquiry using RAG.



3.2.2 Loan Calculation Sequence

This sequence represents how BankWise AI processes a customer request to calculate a car-loan installment.

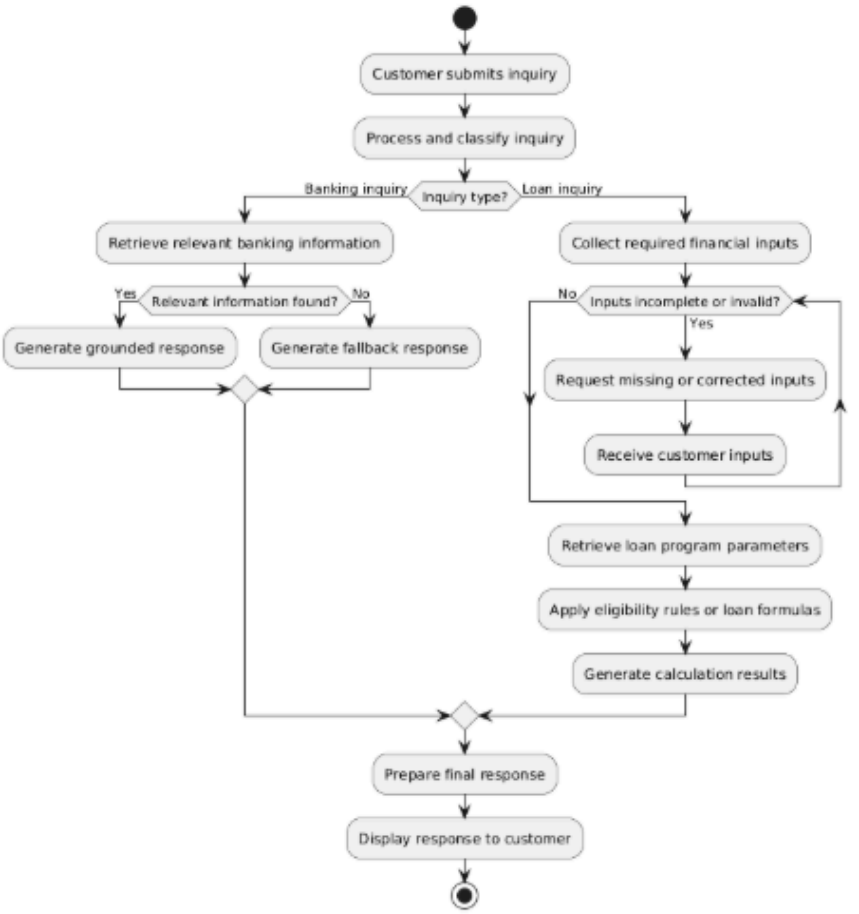


If the customer provides invalid or incomplete inputs, the system requests clarification before performing the calculation.

3.3 Activity Diagram

The activity diagram represents the overall workflow followed when a customer interacts with BankWise AI.

The system first receives and classifies the inquiry, then follows the appropriate processing path depending on whether the request concerns general banking information or loan-related operations.



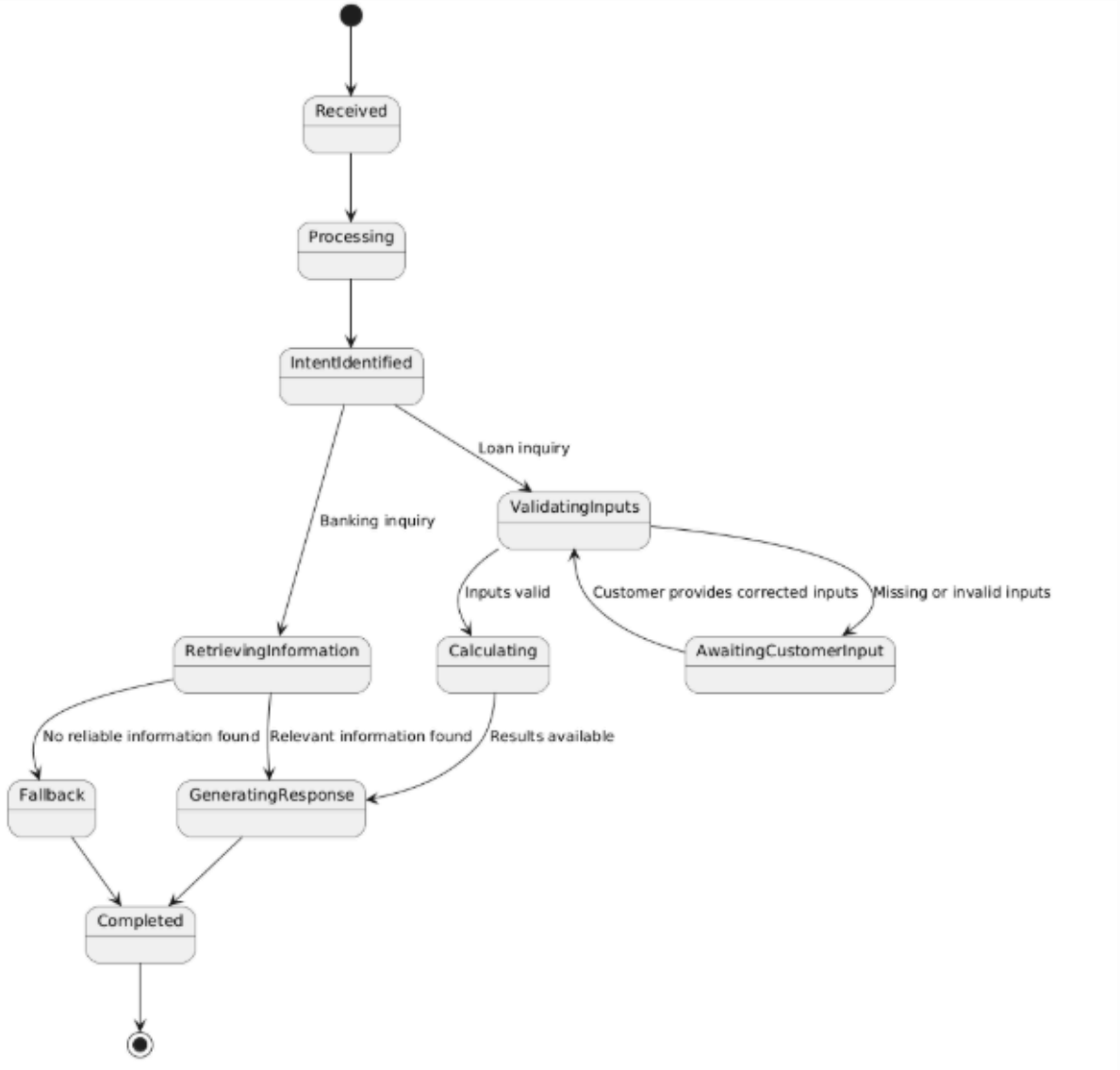
Workflow Summary:

1. The customer submits an inquiry.
2. The AI agent identifies the inquiry type.
3. General banking inquiries are processed through the RAG pipeline.
4. Loan inquiries are processed through the deterministic loan engine.
5. The system validates the availability and correctness of the required information.
6. A response is generated and returned to the customer.

3.4 State Diagram

The state diagram represents the lifecycle of a customer inquiry as it moves through BankWise AI.

The inquiry changes state depending on the processing stage and whether the system has sufficient information to complete the request.



Main States:

- **Received:** The system receives a new customer inquiry.
- **Processing:** The inquiry is being analyzed.
- **Intent Identified:** The system determines the appropriate processing path.
- **Retrieving Information:** The RAG pipeline searches the knowledge base.
- **Validating Inputs:** Financial inputs are checked against required conditions.
- **Awaiting Customer Input:** Additional or corrected information is required.
- **Calculating:** The loan engine performs the required calculations.
- **Generating Response:** The system prepares the final answer.
- **Fallback:** The system cannot retrieve sufficient confirmed information.
- **Completed:** The response has been returned to the customer.

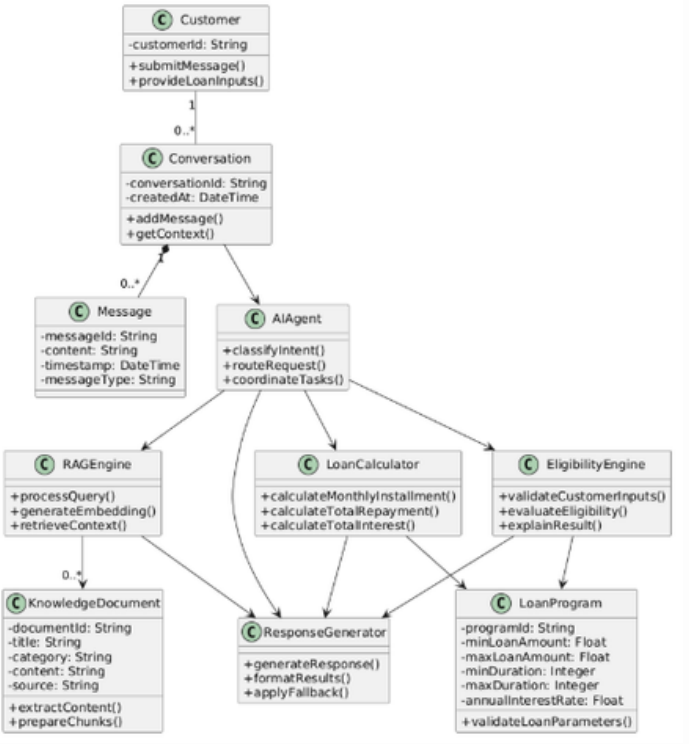
3.5 Class Diagram

The class diagram defines the conceptual structure of BankWise AI by identifying its main classes, attributes, methods, and relationships.

Main Classes:

- **Customer:** Represents the person interacting with the assistant.
- **Conversation:** Maintains a customer's conversation session.
- **Message:** Represents an individual customer message or system response.
- **AI Agent:** Processes inquiries and coordinates the required components.
- **RAG Engine:** Retrieves relevant banking information.
- **Knowledge Document:** Represents a source document used by the RAG pipeline.
- **Loan Program:** Stores the parameters of a car-loan program.
- **Loan Calculator:** Calculates estimated installments and repayment amounts.
- **Eligibility Engine:** Evaluates customer inputs against predefined eligibility rules.
- **Response Generator:** Produces the final customer-facing response.

The diagram represents the logical design of the system rather than its final implementation. Each class has a specific responsibility, allowing the RAG pipeline, loan calculations, and conversational processing to be developed and tested independently.



4. UI/UX DESIGN & PROTOTYPING

4.1 Wireframes & Mockups

BankWise AI uses a web-based interface with a customer conversational banking assistant and a separate staff dashboard. The interface should keep interactions simple, make banking information easy to understand, and present loan calculations clearly.

A. Customer Chat Interface

Main elements: Header with BankWise AI branding; chat area showing customer messages and AI responses; message input; send button; source references for retrieved banking information; structured loan results showing estimated monthly installments and repayment details.

BANKWISE AI – Virtual Banking Assistant
Welcome! How can I help you today?
Customer: What are the available car loans?
BankWise AI: Here are the relevant loan programs and their details...
Source: Banque Misr Loan Information
Customer: Calculate my monthly installment.
BankWise AI: Please provide the loan amount and repayment duration.
Input: Type your question... [Send]

B. Staff Dashboard

The dashboard provides an overview of system usage and operational performance.

Main elements: usage overview (number of customer questions and system requests); performance metrics (response latency and retrieval performance); API usage (token consumption and estimated API costs); recent customer inquiry activity; system monitoring to identify operational issues and frequently asked questions.

Dashboard Area	Displayed Information
Usage Overview	Number of customer questions and system requests
Performance Metrics	Response latency and retrieval performance
API Usage	Token consumption and estimated API costs
Recent Customer Inquiry Activity	Recent questions and response status
System Monitoring	Operational issues and frequently asked questions

Question	Response Status
Car loan inquiry	Completed
Banking fees	Completed
Loan eligibility	Completed
Account services	Completed

System Performance

The System Performance area is intended to present latency and usage metrics, including response latency, retrieval performance, token usage, and estimated API cost. These values are placeholders and do not represent measured results. These wireframes are proposed interface structures for detailed UI prototyping.

4.2 UI/UX Guidelines

Design principles:

1. Simplicity – straightforward interface without unnecessary visual elements.
2. Consistency – consistent layouts, buttons, spacing, and interaction patterns.
3. Clarity – understandable banking information and financial results.
4. Responsiveness – support desktop, tablet, and mobile screens.
5. Feedback – clear processing, success, and error indicators.
6. Trust and Transparency – show relevant information sources and distinguish estimated loan results from official bank decisions.

Role	Specification
Primary	Dark Blue – branding and navigation.
Secondary	White – background and content areas.
Accent	Light Blue – buttons, links, and highlights.
Success	Green – successful operations.
Warning	Orange – warnings and requests requiring attention.
Error	Red – validation errors and failed operations.

Typography

Primary font: Cairo. Use bold headings, regular body text, clear financial result formatting, consistent font sizes and spacing.

Accessibility

Maintain sufficient contrast; use readable font sizes and clear visual hierarchy; provide descriptive labels for inputs and buttons; display validation errors clearly; do not rely on color alone for status; support keyboard navigation; ensure responsive layouts.

5. SYSTEM DEPLOYMENT & INTEGRATION

5.1 Technology Stack

The following technology stack is proposed for the BankWise AI application.

Table 5.1. Proposed technology stack.

Layer	Technology	Purpose
Frontend	HTML, CSS, JavaScript	Customer chat interface and staff dashboard
Typography	Cairo Font	Consistent interface typography
Backend	Python, FastAPI, Uvicorn	Chat API, request orchestration, dashboard API
Retrieval (RAG)	ChromaDB and multilingual embedding model	Search across FAQs, policies, and banking documents
Eligibility	Python rule engine with YAML/JSON configuration	Deterministic eligibility decisions and installment calculations
LLM	External LLM API (provider configurable)	Grounded response generation and loan-input extraction
Data Storage	ChromaDB files, SQLite, JSON documents	Store vectors, metrics, rules, and knowledge-base information
Deployment	Docker Compose on Linux cloud VM	Application hosting and container orchestration
Reverse Proxy	Nginx	HTTPS handling, request forwarding, and rate limiting

Key design decision. Retrieval, conversational processing, and financial calculations are separated to improve reliability and maintainability.

5.2 Deployment Diagram

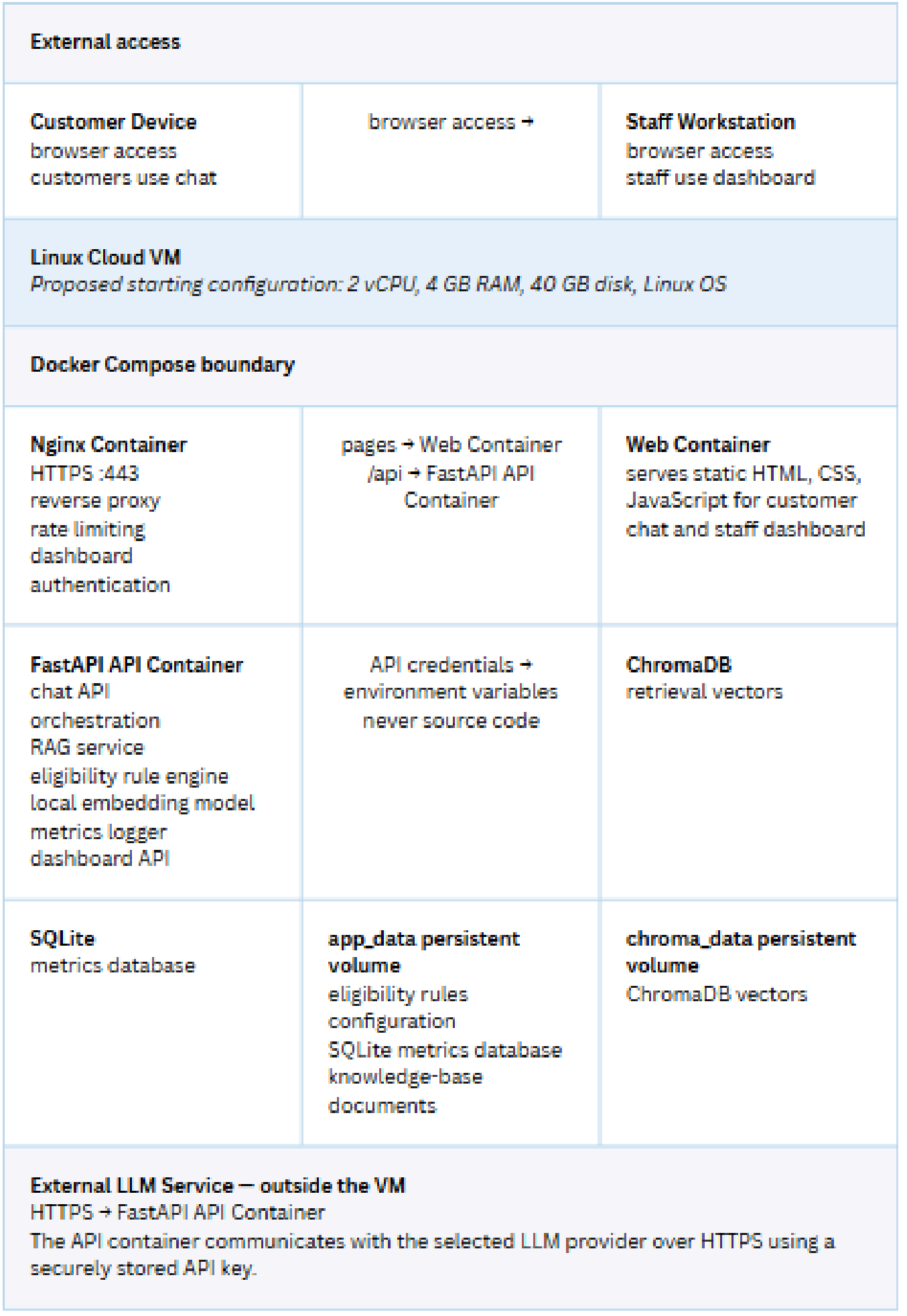
BankWise AI is designed to run on a single Linux cloud virtual machine using Docker Compose.

Proposed VM configuration: 2 vCPU, 4 GB RAM, 40 GB disk, Linux OS. This is a proposed starting configuration, not a tested capacity guarantee.

The deployment design places customer and staff access outside the cloud VM boundary, while the web interface, API, retrieval store, metrics database, and persistent volumes operate within the proposed Docker Compose environment.

Deployment dependencies:

1. **Customer Device and Staff Workstation:** browser access; customers use chat, staff use dashboard.
2. **Nginx Container:** entry point; HTTPS on port 443; rate limiting; dashboard path protected by basic authentication; forwards pages to web container and /api requests to API container.
3. **Web Container:** serves static HTML, CSS, JavaScript for customer chat and staff dashboard.
4. **API Container (FastAPI):** chat API, orchestration, RAG service, eligibility rule engine, local embedding model, metrics logger, dashboard API. API credentials are stored in environment variables, never source code.
5. **Persistent volumes:** chroma_data stores ChromaDB vectors; app_data stores eligibility rules configuration, SQLite metrics database, and knowledge-base documents. These preserve data across container restarts.
6. **External LLM Service:** API container communicates with selected LLM provider over HTTPS using a securely stored API key.



The illustration is a proposed deployment architecture. The VM sizing is a proposed starting configuration and is not a tested capacity guarantee.

5.3 Component Diagram

The application has four layers: Presentation, Application, Data and Models, and Offline Ingestion.

A. Presentation Layer

Customer Chat UI and Staff Dashboard UI. Both communicate with the backend over HTTPS using JSON.

B. Application Layer (FastAPI)

- **Chat API** receives customer messages and forwards them for processing.
- **Input Guard** validates incoming requests, masks sensitive information, and applies request limits.
- **Chat Orchestrator** coordinates processing, maintains session context through Session Manager, and routes inquiries to the appropriate service.
- **Session Manager** maintains temporary conversation context and clears it when the session is reset.
- **RAG Service** retrieves evidence from ChromaDB, generates answers grounded in retrieved information, includes source references, and abstains when sufficient evidence is unavailable.
- **Eligibility Service** extracts and validates financial inputs and applies deterministic eligibility and installment rules.
- **LLM Client** communicates with external LLM provider and handles timeouts, retries, and token-usage capture.
- **Metrics Logger** records response latency, token usage, estimated API cost, and customer questions.
- **Dashboard API** provides metrics to the staff dashboard.

C. Data and Models Layer

- **ChromaDB** stores document embeddings, text chunks, and metadata.
- **Multilingual Embedding Model** converts queries and document chunks into vectors for semantic retrieval.
- **Rules Configuration File** stores loan eligibility rules and calculation parameters in YAML or JSON.
- **Metrics Database (SQLite)** stores operational and usage metrics.
- **Knowledge Base Documents** store structured banking information and source metadata.

D. Offline Ingestion Pipeline

When knowledge base documents change: load updated JSON; normalize and clean content; divide into meaningful chunks; attach metadata including source URL and retrieval date; generate embeddings; store vectors, text chunks, and metadata in ChromaDB. Document preparation remains separate from real-time customer requests.

Layer	Component	Responsibility	Dependency or flow
Presentation	**Customer Chat UI**	Customer conversational interface	HTTPS using JSON → FastAPI application layer
	Staff Dashboard UI	Staff dashboard interface	HTTPS using JSON → FastAPI application layer
Application(FastAPI)	**Chat API**	Receives customer messages and forwards them for processing	→ Chat Orchestrator
	Input Guard	Validates incoming requests, masks sensitive information, and applies request limits	→ Chat API
	Chat Orchestrator	Coordinates processing, maintains session context, and routes inquiries	→ Session Manager; → RAG Service; → Eligibility Service
	Session Manager	Maintains temporary conversation context and clears it when the session is reset	→ Chat Orchestrator
	RAG Service	Retrieves evidence, generates grounded answers, includes source references, and abstains when evidence is insufficient	→ ChromaDB; → Embedding Model; → LLM Client
	Eligibility Service	Extracts and validates financial inputs and applies deterministic eligibility and installment rules	→ Rules Config YAML/JSON; → LLM Client
	LLM Client	Communicates with external LLM provider and handles timeouts, retries, and token-usage capture	→ External LLM provider over HTTPS
	Metrics Logger	Records response latency, token usage, estimated API cost, and customer questions	→ SQLite Metrics DB
	Dashboard API	Provides metrics to the staff dashboard	→ SQLite Metrics DB

Data and Models	**ChromaDB**	Stores document embeddings, text chunks, and metadata	← RAG Service; ← Embedding Generation
	Embedding Model	Converts queries and document chunks into vectors for semantic retrieval	→ ChromaDB; ← RAG Service
	Rules Config YAML/JSON	Stores loan eligibility rules and calculation parameters	→ Eligibility Service
	SQLite Metrics DB	Stores operational and usage metrics	← Metrics Logger; → Dashboard API
	Knowledge Base Documents	Store structured banking information and source metadata	→ Offline ingestion pipeline
Offline Ingestion	**Document Loader**	Loads updated JSON	→ Text Normalization
	Text Normalization	Normalizes and cleans content	→ Chunking
	Chunking	Divides content into meaningful chunks	→ Metadata Preparation
	Metadata Preparation	Attaches metadata including source URL and retrieval date	→ Embedding Generation
	Embedding Generation	Generates embeddings for the prepared chunks	→ ChromaDB

Separation rule: Offline ingestion flows from Document Loader through Embedding Generation to ChromaDB and remains separate from real-time customer requests.

The proposed component architecture separates presentation, real-time application services, data and models, and offline document preparation. Labelled dependencies show how retrieval, eligibility, metrics, and ingestion interact while remaining independently maintainable.

5.4 Integration Overview

1. **Frontend–Backend**: customer chat and staff dashboard communicate with FastAPI through HTTPS JSON requests.
2. **Backend–RAG**: Chat Orchestrator routes banking inquiries to RAG Service, which retrieves information from ChromaDB.
3. **Backend–Eligibility**: loan inquiries are routed to Eligibility Service, which applies configured rules and formulas.
4. **AI Service**: LLM Client communicates with external LLM provider through HTTPS and manages API requests.
5. **Data Integration**: ChromaDB stores retrieval data; SQLite and JSON/YAML files support metrics and structured banking information.
6. **Offline Knowledge Integration**: ingestion pipeline processes updated documents and refreshes ChromaDB.

Conclusion. Modular integration allows RAG, loan engine, and interfaces to be maintained independently while operating as one application.

API Documentation.

1. Introduction

BankWise AI is a 24/7 virtual banking assistant for Egyptian retail banking customers. It combines two engines behind a single chat API:

- **RAG FAQ engine:** answers product, fee and policy questions strictly from retrieved bank documents, with source attribution.
- **Loan eligibility engine:** a deterministic, rule-based calculator (no LLM arithmetic) that returns an eligibility decision, the reasons, and an estimated monthly installment.

This document defines the HTTP API that the React client (and any other consumer) uses to talk to the FastAPI backend. It is written as a design specification: it is the contract that frontend and backend developers build against, and the FastAPI-generated Swagger UI (/docs) should match it.

1.1 Scope

- In scope: chat, session reset, direct loan eligibility check, and health check.
- Out of scope for this version: administrative dashboard, usage analytics, cost/token reporting endpoints, user accounts, and knowledge-base management endpoints. Knowledge-base ingestion is performed offline by a script.

Note on examples: all figures in the examples (fees, limits, rates, amounts) are illustrative placeholders. Real values come from the indexed bank documents and from the credit-policy configuration file.

2. General Conventions

Item	Specification
`Base URL`	http://localhost:8000/api/v1 (development). Replace with the deployed host in production.
`Versioning`	Version is part of the path (/api/v1). Breaking changes require a new version.
`Protocol / format`	HTTPS in production. All requests and responses use JSON (Content-Type: application/json; charset=utf-8).
`Language`	The service is Arabic-only. User-facing text fields in responses (reply, reasons, disclaimer) are in Arabic. Field names and error codes are in English.
`Authentication`	None. All endpoints are open (no admin endpoints exist). Provider API keys (LLM) are stored server-side in environment variables and are never exposed to clients.
`Session handling`	The server generates session_id (UUID v4) on the first /chat call and returns it. The client sends it back on every following call.
`Request tracing`	Every response includes an X-Request-ID header for debugging.
`Character encoding`	UTF-8. Arabic-Indic digits and Arabic number words in messages are normalized server-side.

3. Endpoint Summary

Method	Path	Purpose
POST	`/api/v1/chat`	Send a customer message; receive the assistant reply (FAQ answer, loan-data question, loan result, or fallback).
POST	`/api/v1/session/reset`	Clear the current session and start a new conversation.
POST	`/api/v1/loan/eligibility`	Run the deterministic eligibility check directly. Also called internally by the chat agent.
GET	`/api/v1/health`	Check that the service and its dependencies are running.

4. POST /chat

Main conversational endpoint. The backend loads session memory, decides the intent (FAQ, loan eligibility, out of scope), runs retrieval or the loan tool, and returns a grounded Arabic reply.

4.1 Request body

Field	Type	Required	Description
`session_id`	string (UUID)	No	Omit or send null to start a new session. Send the value returned earlier to continue it.
`message`	string	Yes	Customer message in Arabic (Egyptian dialect supported). 1–1000 characters after trimming.

```
{
  "session_id": null,
  "message": "ايه معايير الإدارة للفرق الشهري؟"
}
```

4.2 Response body (200 OK)

Field	Type	Description
`session_id`	string	Session identifier. Always returned.
`reply`	string	Assistant reply in Arabic.
`intent`	string	One of: faq, loan_collecting, loan_result, no_answer, out_of_scope, unsupported_language, system_fallback (see 4.3).
`sources`	array	Documents used for the answer. Empty for non-FAQ intents. Each item: document (name), url, retrieved_at (ISO date).
`loan`	object | null	Present when intent is loan_collecting or loan_result (see 4.4).
`latency_ms`	integer	Optional. Server processing time. Useful for testing and

Field	Type	Description
		the 5 s KPI.
`usage`	object	Optional. LLM token usage: prompt_tokens, completion_tokens.

4.3 Intent values

intent	Meaning
`faq`	Answer grounded in retrieved documents. sources is non-empty.
`loan_collecting`	The assistant is asking for a missing loan input (one at a time). loan.missing lists what is still needed.
`loan_result`	Eligibility result returned. loan.result is populated and the reply contains the preliminary-estimate disclaimer.
`no_answer`	Retrieval relevance was too low. The reply states that no confirmed information is available and suggests contacting the bank.
`out_of_scope`	Non-banking or unsupported request. Polite refusal and redirect.
`unsupported_language`	Message is not in Arabic. The reply states that the service is currently Arabic-only.
`system_fallback`	The LLM, a tool, or retrieval failed. A clear Arabic fallback message is returned (HTTP 200) instead of a technical error.

4.4 The loan object

Field	Type	Description
`collected`	object	Inputs captured so far, e.g. monthly_income, existing_monthly_debts, loan_amount, tenor_months, age.
`missing`	array	Names of inputs still required. The assistant asks for the first one.
`result`	object | null	Eligibility result (same structure as section 6.2). Only when intent is loan_result.